Nevertheless, here's how to perform bitwise operations in PHP:

```
$allOnes = 0xFF; // int(255) uses 8 bytes for storage on 64-bit systems
print decbin($allOnes) . PHP_EOL;
print decbin($allOnes << 1) . PHP_EOL;
print decbin($allOnes >> 1) . PHP_EOL;
```

```
Output:
11111111
111111110
1111111
```

Here we've allocated a byte value of *0xFF* to `$allOnes`. Hexadecimal digits require four bits because there are sixteen of them. *1111* in binary is *F* in hexadecimal; therefore, *11111111* is *0xFF*, a byte with all bits set to *1*. The << operator is a *left shift*; it shunts all the bits one place to the left and sets the rightmost to zero, resulting in *111111110* (or *00000001 11111110* if you prefer). The inverse is *right-shift* >> and pushes all bits to the right, so we get *1111111* (*01111111*).

Essentially, a single left shift is a binary multiplication by two, and a single right shift is a binary division by two:

```
var_dump($allOnes);
var_dump($allOnes << 1);
var_dump($allOnes >> 1);
```

```
Output:
int(255)
int(510)
int(127)
```

Remember, these are binary integers, not decimal, so the right-shift gives us decimal *127*, not *127.5* (binary division and decimal division are quite different things). We'll cover the gory details of decimal vs. binary and integer vs. floating point in the next section. Another thing to note is how the "sign" bit is preserved. PHP uses IEEE signed integers; the leftmost bit of the most significant byte is used to set a number to positive or negative (and we'll look at this in more detail later too). For now, take a look at the following: